Programa Certified AI-LLM Solution Architect

Proyecto Final AI-LLM Solution Architect

Definiciones

Sesión 5 y Sesión 6

www.bsginstitute.com

Andrés Felipe Rojas Parra

- Ingeniero electrónico con mas de 23 años de experiencia
- PGP Artificial Intelligence & Machine Leaning at Texas University – Austin, TX
- Master en Inteligencia Artificial y Big Data at IEBSchool – Barcelona, Spain
- Estudiante de PGP en Cloud Computing at Texas University - Austin, TX
- Texas Executive Education Program – McCombs School of Business – Decision Science and AI Immersion Program
- Especialista en MLOps de la Universidad de Duke
- Estudiante de la especialización Deep Learning for Healthcare de la Universidad de Illinois



andres.rojas@triskelss.com
@arojaspa
https://www.linkedin.com/in/arojaspa/
+573005906373

CAPÍTULO 3

03

SESIÓN 5

BSG Institute

Despliegue Inicial y Configuración de Infraestructura

Del diseño al sistema real que corre en la nube y responde consultas.

Cloud IaC | Endpoints LLM | Pipeline RAG | Docker/K8s | Health Checks

OBJETIVOS DE LA SESIÓN

¿Qué dominarán al finalizar la Sesión 5?

Desplegar

Configurar la infraestructura cloud con IaC (Terraform/CloudFormation), provisionar el cluster K8s, el vector store y los servicios de red.

Conectar

Implementar los 3 endpoints operativos (/query, /ingest, /health) y conectar el pipeline RAG completo end-to-end con el LLM API.

Containerizar

Construir el Dockerfile multi-stage, docker-compose.yml con health checks, y el Makefile con comandos documentados para operar el sistema.

Validar

Ejecutar pruebas funcionales básicas que confirmen que el pipeline RAG retorna respuestas relevantes con fuentes citadas en el entorno cloud.

Configuración de Infraestructura — Infrastructure as Code

La infra se define en código, se versiona en Git y se aplica de forma reproducible. Nunca se configura manualmente en la consola cloud.

Stack de Infraestructura — Orden de Despliegue

1. **Red y VPC**
   VPC privada · Subnets · Security Groups · Private Link endpoints
2. **Cluster K8s**
   EKS / AKS / GKE · Node groups · RBAC · Calico network policies
3. **Vector Store**
   Pinecone / Qdrant cloud · Namespace · API keys en secrets manager
4. **App Deployment**
   Helm charts · ConfigMaps · Secrets · HPA (auto-scaling)
5. **Observabilidad**
   Langfuse · OpenTelemetry collector · CloudWatch / Datadog

Makefile — Comandos Esenciales del Proyecto

`make setup`	Instala dependencias y configura el entorno local
`make build`	Construye las imágenes Docker multi-stage
`make up`	Levanta el stack completo con docker-compose
`make ingest`	Ejecuta el pipeline de ingestión de documentos
`make test`	Corre la suite completa de pruebas unitarias
`make load-test`	Ejecuta prueba de carga con k6/Locust
`make deploy`	Despliega en cloud vía Helm / kubectl apply
`make health`	Verifica el estado de todos los servicios

Dockerfile Multi-Stage y docker-compose.yml

Dockerfile Multi-Stage — Patrón de Producción

```
# Stage 1: Builder
FROM python:3.11-slim AS builder
WORKDIR /app
COPY requirements.txt .
RUN pip install --no-cache-dir -r requirements.txt

# Stage 2: Production
FROM python:3.11-slim AS production
WORKDIR /app
COPY --from=builder /usr/local/lib /usr/local/lib
COPY src/ ./src/
ENV PYTHONPATH=/app
EXPOSE 8000
HEALTHCHECK --interval=30s --timeout=10s \
  CMD curl -f http://localhost:8000/v1/health
CMD ["uvicorn", "src.main:app",
     "--host", "0.0.0.0", "--port", "8000"]
```

docker-compose.yml — Servicios del Sistema

```
services:
  api:
    build: .
    ports: ["8000:8000"]
    env_file: .env
    healthcheck:
      test: ["CMD","curl","-f",
             "http://localhost:8000/v1/health"]
      interval: 30s
      retries: 3
    depends_on:
      redis:
        condition: service_healthy

  redis:
    image: redis:7-alpine
    healthcheck:
      test: ["CMD","redis-cli","ping"]

  langfuse:
    image: langfuse/langfuse:latest
    ports: ["3000:3000"]
    env_file: .env.langfuse
```

Los 3 Endpoints Obligatorios — Implementación y Health Checks

POST /v1/query

Endpoint principal del sistema RAG

```
@router.post("/v1/query")
async def query(req: QueryRequest):
    # 1. Embed query
    embedding = await
embedder.embed(req.query)
    # 2. Search vector store
    docs = await vs.search(embedding, top_k=5)
    # 3. Build prompt + call LLM
    answer = await llm.generate(req.query,
docs)
    return QueryResponse(
        answer=answer.text,
        sources=docs.metadata,
        tokens_used=answer.usage,
        trace_id=str(uuid4())
    )
```

POST /v1/ingest

Ingestión y actualización de documentos

```
@router.post("/v1/ingest")
async def ingest(files: List[UploadFile]):
    results = []
    for file in files:
        # 1. Parse document
        chunks = chunker.split(await
file.read())
        # 2. Embed chunks
        embeddings = await
embedder.batch(chunks)
        # 3. Upsert to vector store
        ids = await vs.upsert(embeddings)
        results.append({"file": file.filename,
                        "chunks": len(ids)})
    return {"ingested": results}
```

GET /v1/health

Health check de todos los servicios

```
@router.get("/v1/health")
async def health():
    checks = await asyncio.gather(
        vs.ping(),       # vector store
        llm.ping(),      # LLM API
        cache.ping(),    # Redis
        return_exceptions=True
    )
    status = "healthy" if all(
        not isinstance(c, Exception)
        for c in checks) else "degraded"
    return {"status": status,
            "version": settings.VERSION,
            "checks": {...}}
```

S5 · PIPELINE RAG — IMPLEMENTACIÓN

Pipeline RAG End-to-End — Del Documento a la Respuesta

Cada etapa del pipeline debe ser observable, testeada independientemente y tolerar fallos sin romper el sistema completo.

1 Ingestión

PyMuPDF / python-docx · Chunking recursivo · Metadata preservada

2 Embedding

OpenAI text-embedding-3-large · Batch async · Rate limit handler

3 Indexado

Pinecone upsert · Namespace por tenant · Metadata filterable

4 Recuperación

Cosine similarity · top-k=5 · Threshold 0.78 · MMR re-ranking

5 Augmentación

Context window management · Source formatting · Token counting

6 Generación

Streaming response · Retry con backoff · Fallback a cache

Pipeline CI/CD — GitHub Actions en Verde

GitHub Actions — Flujo de CI/CD

🔍 **Lint & Type Check** ruff · mypy · bandit	on: push
🧪 **Unit Tests** pytest · coverage ≥ 60%	paralelo
🐳 **Build Docker** docker buildx · multi-arch	si tests ✓
🔐 **Security Scan** pip-audit · gitleaks	paralelo
🧩 **Integration Test** pytest RAG end-to-end	si build ✓
🚀 **Deploy to Cloud** kubectl apply / helm upgrade	si todo ✓

Quality Gates — Qué Debe Pasar Para Desplegar

✓ Cobertura de pruebas ≥ 60%

✓ 0 errores de tipo (mypy --strict)

✓ 0 hallazgos de seguridad alta/media

✓ 0 credenciales en el código (gitleaks)

✓ Prueba de integración RAG pasa

✓ /health retorna 200 post-deploy

✗ ruff reporta errores de estilo

✗ print() como logging en producción

Pruebas Funcionales — Validación del Pipeline RAG

Una prueba funcional no es 'probé y respondió bien' — es un test automatizado con assertions medibles contra los RNF definidos.

test_rag_pipeline.py — Prueba de Integración Obligatoria

```python
@pytest.fixture
def client():
    return TestClient(create_app())

def test_query_returns_relevant_answer(client):
    payload = {
        "query": "¿Cuál es la política de vacaciones?",
        "session_id": str(uuid4())
    }
    start = time.time()
    response = client.post("/v1/query", json=payload)
    latency = time.time() - start

    # Assertions contra los RNF definidos
    assert response.status_code == 200
    assert latency < 3.0, f"p95 violated: {latency:.2f}s"

    data = response.json()
    assert len(data["answer"]) > 50
    assert len(data["sources"]) >= 1
    assert data["sources"][0]["score"] >= 0.78
    assert "trace_id" in data

def test_health_all_services_up(client):
    r = client.get("/v1/health")
    assert r.status_code == 200
    assert r.json()["status"] == "healthy"
```

Anti-Patrones de Testing

✗ Probar solo manualmente desde Postman

✗ Assertions solo sobre status 200

✗ Tests sin mocks que llaman APIs de pago

✗ No medir latencia en las pruebas

✗ Tests que dependen de orden de ejecución

✗ Happy path únicamente, sin casos de error

✓ Fixtures reales ✓ Mock LLM en unit tests ✓ Latencia medida

Resumen Sesión 5

¿Qué construiste hoy?



Infraestructura cloud
con IaC desplegada



Dockerfile + compose
con health checks



3 endpoints operativos
en producción



Suite de pruebas
funcionales corriendo

PRÓXIMO: Sesión 6 — Optimización y Pruebas de Carga

Medición de rendimiento · Evaluación LLM con RAGAS · Pruebas de carga · Cumplimiento y Ética

CAPÍTULO 3

03

SESIÓN 6

BSG Institute

Optimización, Pruebas de Carga y Cumplimiento

Un sistema que funciona no es un sistema de producción — necesita escalar, ser ético y cumplir normas.

RAGAS Eval | Prueba de Carga | Seguridad | Ética AI | Entregable E3

Evaluación de Calidad LLM — Framework RAGAS

RAGAS evalúa cuantitativamente la calidad del sistema RAG. 'El sistema respondió bien' no es evidencia — los scores sí lo son.

Las 5 Métricas Clave de RAGAS

≥ 0.85	**Faithfulness**	¿La respuesta está fundamentada en el contexto recuperado? Detecta alucinaciones.
≥ 0.80	**Answer Relevancy**	¿La respuesta responde lo que el usuario preguntó? Penaliza divagaciones.
≥ 0.75	**Context Recall**	¿El retriever recuperó los fragmentos necesarios para responder correctamente?
≥ 0.78	**Context Precision**	¿Los fragmentos recuperados son relevantes? Penaliza ruido en el contexto.
≥ 0.80	**Answer Correctness**	¿La respuesta es factualmente correcta comparada con el ground truth?

ragas_evaluation.py — Implementación

```python
from ragas import evaluate
from ragas.metrics import (
    faithfulness, answer_relevancy,
    context_recall, context_precision,
    answer_correctness
)

# Dataset: ≥ 20 pares Q/A del caso de uso
dataset = load_dataset("./data/eval_dataset.json")

result = evaluate(
    dataset,
    metrics=[
        faithfulness,
        answer_relevancy,
        context_recall,
        context_precision,
        answer_correctness,
    ]
)

# Output: reports/ragas_report.json
result.to_pandas().to_json(
    "reports/ragas_report.json"
)
print(result)
# faithfulness      : 0.87
# answer_relevancy  : 0.83
# context_recall    : 0.79
```

Prueba de Carga — Midiendo p50, p95, p99 y Throughput

k6 Load Test Script — load_test.js

```
import { check, sleep } from 'k6';
import { Rate, Trend } from 'k6/metrics';

const latencyTrend = new Trend('query_latency');
const errorRate = new Rate('error_rate');

export const options = {
  stages: [
    { duration: '2m', target: 10 },  // ramp up
    { duration: '5m', target: 50 },  // sustained
    { duration: '2m', target: 0  },  // ramp down
  ],
  thresholds: {
    'query_latency': ['p(95)<3000'], // RNF
    'error_rate':    ['rate<0.01'],  // <1% errors
    'http_req_duration': ['p(50)<1200'],
  },
};

export default function () {
  const res = http.post(
    'https://api.tu-sistema.com/v1/query',
    JSON.stringify({ query: randomQuery() }),
    { headers: { 'Content-Type': 'application/json',
                 'Authorization': `Bearer ${__ENV.TOKEN}` } }
  );
  latencyTrend.add(res.timings.duration);
  errorRate.add(res.status !== 200);
  check(res, { 'status 200': r => r.status === 200 });
  sleep(1);
}
```

Reporte de Resultados — Contra los RNF

✓	p50 latencia:	1,180ms
✓	p95 latencia:	2,840ms
⚠	p99 latencia:	4,200ms
✓	Throughput:	18 RPS
✓	Error rate:	0.3%

Estrategias de Optimización

- → Semantic caching: respuestas frecuentes en Redis
- → Embedding cache: no re-embedear la misma query
- → Streaming: TTFT < 800ms aunque respuesta tome 4s
- → HPA: escalar pods bajo carga sostenida > 15 RPS
- → top-k reducido a 3 para queries simples vs 5

S6 · SEGURIDAD — IMPLEMENTACIÓN

Seguridad en Código — Gestión de Secretos y Autenticación

Un solo API key commiteado al repositorio invalida semanas de trabajo de seguridad. La gestión de secretos es la primera línea de defensa.

Gestión de Secretos

- Variables sensibles SOLO en .env (no en el repo)
- En producción: AWS Secrets Manager / Azure KV
- .gitignore con .env, *.key, credentials*
- gitleaks en CI bloquea push con secretos
- Rotación automática de API keys

Autenticación JWT

- JWT firmado con RS256 (clave asimétrica)
- Expiración 15 min + refresh token 24h
- Middleware de auth en TODOS los endpoints
- Rate limiting: 100 req/min por usuario
- Logs de autenticación fallida con IP

Escaneo de Seguridad

- bandit: análisis estático de código Python
- pip-audit: dependencias con CVEs conocidos
- 0 hallazgos de severidad MEDIA o ALTA
- Reporte en reports/security_scan.json
- Ejecutado en CI en cada pull request

Regla de oro: Si está en el repositorio Git, asume que es público — aunque sea un repo privado.

S6 · CUMPLIMIENTO Y ÉTICA AI

Dimensión Ética y de Cumplimiento en Sistemas AI/LLM

Los sistemas AI tienen riesgos éticos específicos que no existen en software tradicional. El arquitecto los identifica y mitiga desde el diseño.

Alucinaciones

El LLM puede inventar información con confianza. Mitigación: Faithfulness ≥ 0.85 medido con RAGAS, respuesta solo basada en contexto recuperado, fuentes citadas obligatorias.

Sesgos del Modelo

Los LLMs heredan sesgos del corpus de entrenamiento. Mitigación: Dataset de evaluación diverso, revisión manual de respuestas sensibles, alerta cuando el modelo aplique juicios de valor.

Transparencia

Los usuarios tienen derecho a saber que interactúan con AI. Mitigación: Disclosure explícito en UI/API, fuentes citadas en cada respuesta, trace_id para auditoría.

Privacidad de Datos

El contexto RAG puede contener PII. Mitigación: PII scrubbing antes del embedding, namespace por tenant, retención configurada, DSAR implementado.

Equidad

El sistema no debe responder diferente a usuarios por características protegidas. Mitigación: Test de equidad en el dataset de evaluación, revisión de casos límite.

Human-in-the-Loop

Para decisiones de alto impacto (créditos, diagnósticos), el AI no es el decisor final. Mitigación: Flag de confianza < 0.8, escalamiento a humano, registro de decisiones.

Ajuste de Costos — El LLM No Es el Único Gasto

Desglose de Costo Mensual MVP — Ejemplo Real



Palancas de Optimización de Costo

-30%	**Semantic Caching** — Redis cache de respuestas frecuentes. ROI: <1 semana.
-40%	**Modelo más pequeño** — GPT-4o-mini para queries simples, GPT-4o solo para complejas.
-15%	**Prompt compresión** — Reducir contexto de 5 a 3 chunks cuando score es alto.
-60%	**Spot instances** — Workloads de ingestión batch en nodos spot/preemptible.
-20%	**Embedding cache** — No re-embeber queries repetidas en 24h.
∞	**Alertas de budget** — Alerta cuando costo diario supera $35. Evita sorpresas.

ACTIVIDAD PRÁCTICA EN CLASE

Taller Integrador — Sesión 6: 45 minutos

Con el sistema del caso financiero (empresa 500 emp., 2K consultas/día) ya desplegado en S5, ahora miden, optimizan y evalúan:

15 min

1. Evaluación RAGAS

Construyan un dataset de ≥ 10 pares Q/A representativos del caso (consultas sobre inversiones, seguros, crédito). Corran RAGAS y reporten faithfulness, answer_relevancy y context_recall. ¿Qué métrica es la más baja? ¿Por qué?

10 min

2. Prueba de Carga k6

Ejecuten una prueba de 5 minutos con 10 usuarios concurrentes. Reporten p50, p95 y error rate. ¿El sistema cumple los RNF definidos en S1? Si no cumple, identifiquen el cuello de botella (embedding, vector search, o LLM generation).

10 min

3. Análisis Ético

Revisen 5 respuestas del sistema e identifiquen: ¿hay alguna que podría considerarse sesgada? ¿hay alguna donde el LLM haya inventado información no presente en los documentos? ¿cómo lo mitigan?

10 min

4. Presentar Resultados

Cada equipo comparte su score RAGAS más bajo y su cuello de botella de rendimiento. El instructor facilita: ¿Es mejor bajar top-k para mejorar precisión? ¿Vale la pena el semantic caching dado el volumen?

ENTREGABLES CAPÍTULO 3

Entregable 3 — Implementación Funcional (Cap. 3)

Peso como hito: 30 puntos · Sesiones S5–S6 · Entrega: Fin de S6

✓ OBL	3.1	Código fuente completo y funcional en src/
✓ OBL	3.2	Los 3 endpoints operativos: /query, /ingest, /health
✓ OBL	3.3	Pipeline RAG funcional end-to-end (demostrable)
✓ OBL	3.4	Dockerfile multi-stage + docker-compose.yml con health checks
✓ OBL	3.5	Sistema desplegado en cloud con URL pública en el README
✓ OBL	3.6	Suite de pruebas unitarias con cobertura ≥ 60% (coverage.xml)
✓ OBL	3.7	Al menos 1 prueba de integración del pipeline RAG end-to-end
✓ OBL	3.8	
✓ OBL	3.9	Prueba de carga con reporte (≥ 10 usuarios concurrentes)
✓ OBL	3.10	Pipeline CI/CD operativo y en verde en GitHub Actions
✓ OBL	3.11	Makefile con comandos documentados
✓ OBL	3.12	Secciones 6 y 7 de la Plantilla Oficial completadas
⚠ Rec.	3.13	Reporte de escaneo de seguridad (bandit + pip-audit)
⚠ Rec.	3.14	Cobertura de pruebas ≥ 80%
⚠ Rec.	3.15	Dataset de evaluación LLM con ≥ 20 pares Q/A en notebooks/

Todos los entregables se suben al repositorio Git. Sistema debe estar desplegado y accesible con URL pública. Tardíos: -20%.

CRITERIOS DE EVALUACIÓN

D3, D4 y D5 — Criterios de Evaluación del Capítulo 3 (36 pts)

Propósito: Evalúa implementación técnica, seguridad/ética y rigurosidad en pruebas del sistema AI/LLM.

D3.1 Funcionalidad del Sistema — 7 pts

MAX: 3 endpoints en cloud. Pipeline RAG retorna respuestas con fuentes citadas. Manejo de errores graceful (modelo no disponible, vector store vacío, input inválido).

MED: 3 endpoints funcionan. RAG opera. Manejo de errores básico presente.

LOW: Sistema parcial o solo local.

D3.2 Calidad del Código — 7 pts

MAX: SOLID, responsabilidad clara, bajo acoplamiento. Manejo errores explícito con excepciones tipadas. Type hints en todas las funciones públicas. Pasa ruff y mypy sin errores.

MED: Código bien organizado. Manejo de errores existe aunque no exhaustivo.

LOW: Notebook convertido a script sin estructura. Credenciales hardcodeadas.

D3.3 Reproducibilidad — 4 pts

MAX: Reproducible desde README en < 15 min en máquina limpia. Dockerfile multi-stage. docker-compose con health checks. Makefile cubre todos los flujos.

MED: Reproducible con algunos pasos adicionales no documentados. Docker funciona.

LOW: Solo corre localmente. README incompleto.

D4.1 Gestión de Secretos — 3 pts

MAX: 0 credenciales en repo (gitleaks). Variables en .env + secrets manager en prod. JWT robusto. Rate limiting. Bandit + pip-audit sin hallazgos media/alta.

MED: Sin credenciales expuestas. Auth existe y funciona. Rate limiting puede estar ausente.

LOW: Credenciales expuestas en código o historial Git.

D4.2 Modelo de Amenazas AI/LLM — 4 pts

MAX: ≥ 6 amenazas con nivel de riesgo, control implementado y evidencia. Cubre prompt injection, data leakage via LLM, hallucinations, jailbreaking. Guardrails implementados.

MED: ≥ 4 amenazas con controles. Amenazas críticas AI/LLM identificadas.

LOW: Solo amenazas genéricas de API sin las específicas de AI/LLM.

D5 Pruebas y Validación — 8 pts

MAX: Cobertura ≥ 80%. ≥ 1 integración RAG E2E. RAGAS con ≥ 5 métricas vs umbrales. Dataset ≥ 20 Q/A. Prueba de carga con ≥ 50 usuarios. p50, p95, p99 reportados.

MED: Cobertura ≥ 60%. ≥ 1 integración. RAGAS con ≥ 3 métricas. Carga con ≥ 10 usuarios.

LOW: Pruebas manuales sin métricas. No se usa framework de evaluación.

¿Qué aprendiste en el Capítulo 3?



Infraestructura cloud con IaC y CI/CD



3 endpoints RAG en producción



Evaluación RAGAS y pruebas de carga



Seguridad, secretos y ética AI

PRÓXIMO: Capítulo 4 — Documentación y Defensa

Informe técnico final · Documentación arquitectónica completa · Presentación ejecutiva · Defensa con panel evaluador

Recordatorio — Entregable 3

src/ completo · 3 endpoints en cloud · Dockerfile + CI/CD · Suite de pruebas ≥ 60% cobertura · RAGAS report · Prueba de carga · Bandit/pip-audit → repositorio Git antes de S7